

CODEFEST AD ASTRA 2026 — Etapa 1: Informe Técnico

Nota: Las secciones numéricas que se citan a lo largo del documento (por ejemplo, sección 3.3), son las secciones del documento “CODEFEST_2026-1”, el cual contiene el enunciado del reto.

1. Resumen

Se construyó una base de conocimiento vectorial a partir del corpus completo provisto por ADL (1,846 archivos, 3.0 GB, formatos PDF/JSON/CSV/XLSX/imágenes/PBF) para los tres fenómenos del reto. El pipeline completo (extracción -> limpieza -> chunking -> codificación -> indexación -> recuperación) se puede ejecutar mediante los scripts incluidos y *generador.py*.

- **Documentos extraídos:** 1,670 (de 1,846 archivos vistos; ver Sección 2 para el detalle de las 175 exclusiones)
- **Fragmentos indexados:** 92,732
- **Encoder:** *intfloat/multilingual-e5-base* (768 dim)
- **Índice:** FAISS *IndexFlatIP* sobre vectores normalizados (similitud coseno)
- **Distribución por fenómeno (documentos):** F1=434, F2=466, F3=770
- **Distribución por idioma (fragmentos):** EN=70,648 (76%), ES=21,762 (23%), PT=322 (0.3%)

2. Preprocesamiento y decisiones de exclusión

El corpus se procesó archivo por archivo según su formato (Sección 2: PDF vía PyMuPDF, JSON con ramas por familia de esquema, CSV/XLSX fila por fila, HTML/TXT con limpieza de boilerplate). Cada archivo recibió un *doc_id* incremental y un campo *fuentes* con su ruta relativa dentro del corpus (única, usada como clave de emparejamiento con el *ground truth* según la Sección 10.2.1).

Se documentan explícitamente las siguientes exclusiones deliberadas (registradas en *data/exclusions.jsonl*, 176 archivos en total), motivadas por evitar diluir el índice con contenido no informativo o ajeno a los tres fenómenos, sin sacrificar cobertura real:

Categoría	Archivos	Razón
Tiles vectoriales <i>.pbf</i> (Amazon Underworld)	73	Duplican, tile por tile y nivel de zoom, el mismo contenido ya disponible íntegro en <i>AMAZONUW_amazonunderworld-data.csv</i> (4,370 filas, sí indexado)
Sin contenido extraíble	57	45 son PDF escaneados sin capa de texto (serie <i>Alertas Tempranas/Informes</i> ; el contenido de cada alerta individual sí está cubierto por los 363 JSON estructurados de <i>alertas/</i>); el resto son

		JSON/PDF genuinamente vacíos
Datasets bibliométricos crudos del AI Index	21	<i>recursos/{Healthcare_Medicine, Research_Development}/datasets/</i> : listados de PubMed, ensayos clínicos y Microsoft Academic Graph (algunos >100,000 filas) que alimentan gráficas específicas del reporte, ajenos a los 3 fenómenos
Archivos de sistema (.DS_Store)	9	Basura de macOS no Finder, no son documentos
Imágenes de portada (jpg/avif)	9	Sin OCR en esta entrega; en su mayoría portadas de reportes ya indexados en texto vía su PDF/JSON asociado
Categoría	Archivos	Razón
Duplicados de formato (mismo dataset en .csv y .json)	6	Catálogos de RUTAN, DAIO, CSIS, RESDAL, MAPP-OEA, SWF: se conserva solo .csv para no ocupar dos veces un cupo de los 3 documentos en F1@3
Total	175	

La exclusión de los datasets bibliométricos crudos es la de mayor impacto en volumen: su inclusión habría multiplicado el índice entre 10 y 50 veces (un solo archivo, *pubmed-artificial-intelligence-csv.csv*, tiene 111,776 filas) sin aportar señal útil para ninguna de las 50 consultas de evaluación. La narrativa completa de cada reporte anual del AI Index (2017–2026) se conserva vía sus PDF (*recursos/*/pdfs/*).

3. Estrategia de chunking

Se usa una estrategia híbrida jerárquica + por oración con superposición (Sección 3.2):

- 1 Cada extractor produce una secuencia de bloques tipados (*heading/para/row*) que preserva la estructura reconocible del documento origen (encabezados por tamaño de fuente en PDF, secciones en JSON, filas en CSV/XLSX).
- 2 Los bloques de prosa se segmentan en oraciones (*pysbd*, con detección de idioma ES/EN/PT por documento vía *langdetect*).
- 3 Las oraciones/filas se agrupan hasta un objetivo de 350 tokens (límite duro de 450), dejando margen bajo el límite de 512 tokens del encoder. El corte se prefiere justo antes de un nuevo encabezado, salvo que el chunk acumulado sea demasiado pequeño (<40 tokens) para ser autocontenido.

- 4 Ninguna oración se corta a la mitad (Sección 3.3): si una sola unidad (oración o fila) excede el máximo, se trunca al límite del encoder en vez de generar un chunk desproporcionado (ocurre en 124 casos, 0.13% de los chunks, concentrados en tablas presupuestales de PDFs institucionales sin puntuación oracional real y en texto decorativo repetido de portadas). El encoder truncaría ese exceso de todas formas al codificar, así que conservarlo completo solo inflaría la metadata sin aportar señal adicional.
- 5 Los chunks nuevos repiten la última oración de prosa del chunk anterior como solapamiento, salvo que el nuevo chunk comience con un encabezado (en una nueva sección, el solapamiento no aporta).
- 6 Las filas de datasets tabulares (CSV/XLSX, catálogos JSON tipo lista de objetos) se tratan como unidades atómicas (no son prosa, no se segmentan en oraciones) pero sí se empaquetan varias filas consecutivas por chunk hasta el mismo objetivo de tokens, para producir fragmentos temáticamente más ricos que una fila aislada (por ejemplo, varios municipios contiguos del dataset de Amazon Underworld en un mismo fragmento).

El campo *texto* almacenado es siempre una concatenación literal de oraciones/filas del documento origen, sin alteraciones (Tabla 1).

4. Codificación semántica: selección del encoder

Se evaluaron los criterios de la Sección 4.3 para elegir un único encoder multilingüe (ver decisión sobre múltiples encoders más abajo):

Criterio	<i>intfloat/multilingual-e5-base</i>
Soporte multilingüe	Nativo en alrededor de 100 idiomas incluyendo. ES/EN/PT (los tres del corpus)
Dimensionalidad	768 - balance entre expresividad y costo de almacenamiento/búsqueda
Longitud máxima	512 tokens (XLM-RoBERTa), consistente con el diseño del chunking
Benchmarks	Fuerte desempeño en recuperación densa en MTEB/BEIR multilingüe
Licencia	MIT
Eficiencia	278M parámetros; corre cómodamente en GPU de 4GB (RTX 3050) o CPU

E5 requiere prefijar "*query:* " / "*passage:* " al texto de entrada; esto se aplica de forma consistente en indexación y recuperación.

Decisión sobre múltiples encoders: se optó por un único encoder en lugar de combinar varios mediante RRF/CombSUM (Sección 8.4). Un segundo encoder duplicaría el tiempo de cómputo y el espacio en disco a cambio de una ganancia marginal esperada, dado que *multilingual-e5-base* ya cubre de forma nativa los tres idiomas del corpus con buen desempeño en recuperación densa. Esta decisión prioriza simplicidad y reproducibilidad sobre una mejora incierta.

5. Índice vectorial FAISS

Se usa *IndexFlatIP* sobre vectores normalizados a norma unitaria (equivalente a similitud coseno, Sección 8.2), tal como recomienda el propio spec para el volumen de documentos esperado en este reto (Sección 5.2). Con 92,732 vectores de 768 dimensiones, una búsqueda exhaustiva es exacta y suficientemente rápida (sin necesidad de *IndexIVFFlat* o *IndexHNSW*, que introducirán aproximación innecesaria a este volumen).

La metadata (Tabla 1) se persiste en *metadata.jsonl*, con el orden de líneas alineado exactamente al id interno asignado por FAISS en el momento de la inserción (Sección 5.3).

6. Módulo de recuperación

Dado el vector de consulta (mismo encoder, prefijo "*query:* "), FAISS devuelve los $k=100$ fragmentos más similares por coseno. En especial, se obtiene la siguiente información:

- **Documentos (top 3):** agregación por *max pooling* sobre *doc_id* (Sección 8.6) — la puntuación de un documento es la de su fragmento más relevante. Se prefirió *max pooling* sobre suma/promedio ponderado porque no penaliza documentos cortos ni favorece artificialmente documentos con muchos fragmentos recuperados.
- **Fragmentos (top 10):** se aplican las reglas de la Sección 9.2.1 — los fragmentos de más de 250 palabras se dividen respetando límites oracionales (todas las sub-partes comparten el *chunk_id* original, cada una con su propio rank); los fragmentos de menos de 125 palabras se enriquecen opcionalmente concatenando el fragmento siguiente del mismo documento por posición si el resultado no supera las 250 palabras.

No se usa ningún modelo generativo/decoder en ninguna etapa (Sección 8.3): ni para reranking, ni para expansión de consulta, ni para síntesis del resultado. Toda la recuperación opera sobre vectores, puntuaciones de similitud coseno y metadata.

Post-filtros: el sistema soporta filtrar por *fenomeno/formato/ umbral mínimo de similitud* (Sección 8.7), pero se deshabilitan por defecto, puesto que inferir el fenómeno de una consulta para filtrar introduciría un supuesto frágil que podría descartar documentos relevantes cross-fenómeno; se prefiere confiar en la señal semántica del encoder multilingüe sobre el corpus completo.

7. Grafo de conocimiento

Se implementó el componente opcional descrito en la Sección 7 del enunciado, construyendo un grafo de conocimiento dirigido $G = (E, R, T)$ a partir de los mismos 92,732 fragmentos indexados en la base vectorial, e integrándolo como señal complementaria en la recuperación.

Reconocimiento de entidades (NER)

Se usó *Davlan/xlm-roberta-base-ner-hrl*, un modelo NER multilingüe (10 idiomas, incluyendo ES/EN/PT) con licencia AFL-3.0 (permisiva; se descartó la alternativa *WikiNEuRal* por su licencia CC BY-NC-SA, no apta para uso comercial). Comparte el mismo backbone *XLM-RoBERTa* que el encoder

multilingual-e5-base usado en la base vectorial, por lo que los chunks del pipeline (≤ 450 tokens) quedan cómodamente bajo su límite de 512 tokens. Las entidades PER/ORG/LOC extraídas por el modelo se complementan con un gazetteer propio de tipo CONCEPTO (alrededor de 75 términos translingües del dominio de los tres fenómenos del reto: IA, basura espacial, LEO, deforestación, entre otros), necesario porque el modelo NER no cubre conceptos técnicos ni fenómenos como entidades por sí solo.

Extracción de relaciones (RE)

Las relaciones entre entidades se extraen mediante análisis de dependencias sintácticas universales (UD) con *spaCy*, identificando patrones sujeto - verbo - objeto (incluyendo voz pasiva) dentro de cada fragmento, con los verbos transformados a su forma en español para unificar relaciones equivalentes expresadas en distintos idiomas del corpus. Consistente con la decisión de la Sección 8.3 de no usar modelos generativos en ninguna etapa del sistema, esta extracción es puramente heurística/sintáctica, sin decoders.

Para los chunks de naturaleza tabular (CSV, y JSON cuando se identifica como catálogo de pares clave-valor, criterio que fue validado sobre los 3,480 archivos JSON del corpus: 68 catálogos frente a 3,412 artículos de prosa) no se aplica parsing sintáctico, dado que no son prosa; en su lugar, las relaciones se derivan por co-ocurrencia de entidades dentro del mismo chunk.

Construcción e integración del grafo

Las tripletas extraídas se almacenan en una estructura *NetworkX* en memoria y se persisten como *grafo.graphml*, junto con *grafo_stats.json* (conteo de nodos, aristas, tipos de entidad y entidades más frecuentes). Cada triplete conserva su *chunk_id* de origen como evidencia trazable (*doc_id* asociado vía *metadata.jsonl*), y cada nodo del grafo mantiene la lista de ids *FAISS* de los chunks donde aparece la entidad correspondiente, cumpliendo la vinculación descrita en la Sección 7.3.

El grafo se construyó sobre el corpus completo de 92,732 chunks en aproximadamente 45 - 70 minutos (NER en GPU RTX 3050 = 15 - 25 min; extracción de relaciones en CPU = 25 - 40 min, paralelizada en 4 procesos).

Integración con la recuperación

En la etapa de recuperación (Sección 8.5), la consulta se procesa con el mismo componente de NER para identificar sus entidades, se expanden los vecinos de primer orden de dichas entidades en el grafo, y el ranking resultante se fusiona con el de la búsqueda vectorial *FAISS* mediante *Reciprocal Rank Fusion* (*RRF*, $k_0=60$). Este comportamiento es opcional y configurable (*USE_GRAPH_FUSION*); si *grafo.graphml* no está presente, el sistema se degrada automáticamente a recuperación solo-vectorial, sin afectar la reproducibilidad del núcleo obligatorio descrita en la Sección 11.

8. Reproducibilidad

generador.py es autocontenido: carga *index.faiss* + *metadata.jsonl*, descarga el mismo encoder desde HuggingFace, lee *consultas.jsonl* y produce *resultados.jsonl* aplicando exactamente la lógica descrita en la Sección 6. Las dependencias están fijadas en *requirements.txt*.

9. Resultados de validación

- **Distribución de tokens por chunk (n=92,732):** mínimo 3, percentil 25 = 136, mediana = 319, percentil 95 = 433, máximo = 502 (tope de seguridad). Ningún chunk excede el límite real del encoder (512 tokens).
- **Tiempos de pipeline (equipo con GPU NVIDIA RTX 3050 4GB):** extracción = 14 min (844s), chunking = 13 min (795s), codificación e indexación = 35 min (2,099s para 92,732 vectores). Total = 62 min de extremo a extremo, sin contar el tiempo de desarrollo/iteración del pipeline.
- **Chequeos de integridad:** 1,670 *doc_id* y *fuelle* únicos sin duplicados; 92,732 *chunk_id* únicos sin duplicados; cero fragmentos con texto vacío; consistencia *doc_id* ↔ *fuelle* verificada en el 100% de los chunks; *resultados.jsonl* valida contra el esquema completo de la Sección 9.3.2 (50 líneas, 3 documentos y 10 fragmentos por línea, ≤250 palabras) (*scripts/scripts/inspect_pipeline.py* y *scripts/validate_resultados.py*).
- **Inspección cualitativa:** se revisó manualmente el resultado de una muestra de consultas reales cubriendo los tres fenómenos (p.ej. q001, q005, q018, q026, q033, q042, q050), verificando que los documentos y fragmentos recuperados fueran temáticamente coherentes con cada pregunta (por ejemplo, una consulta sobre dependencia tecnológica de IA en Colombia recupera en primer lugar la ficha país de Colombia del reporte ILIA 2025 con su puntaje específico; una consulta sobre capacidades contraespaciales recupera los reportes de Secure World Foundation). No se identificaron recuperaciones fuera de tema en la muestra revisada.

10. Guía de replicabilidad

Para validar que la entrega es reproducible en un ambiente distinto al de desarrollo, se definió un procedimiento de prueba en Kaggle. Se quiso comprobar que, a partir únicamente de la carpeta *entrega/*, es posible instalar las dependencias y ejecutar *generador.py* sobre la base vectorial ya construida, reproduciendo *resultados.jsonl* sin modificaciones manuales.

Para esto, se envía solo la carpeta *entrega/* (sin *.venv*, modelos descargados ni corpus original), conservando la estructura:

entrega/

└─ *base_vectorial/encoder_multilingual-e5-base/{index.faiss, metadata.jsonl}*

└─ *grafo/{grafo.graphml, grafo_stats.json}*

|— *consultas.jsonl*

|— *generador.py*

|— *resultados.jsonl*

|— *informe_tecnico.pdf*

|— *requirements.txt*

- **Procedimiento:** En un Kaggle Notebook limpio, se carga el ZIP como Dataset, se instalan las dependencias especificadas en requirements.txt y se ejecuta python generador.py sin realizar modificaciones al código ni instalar dependencias adicionales. Previamente, se conserva una copia del resultados.jsonl original para permitir la posterior comparación con el archivo generado por el script.
- **Criterios de éxito:** el script termina sin errores; se genera/actualiza resultados.jsonl con 50 líneas (q001–q050); cada línea cumple la estructura esperada (3 documentos y 10 fragmentos por consulta); y el archivo generado coincide con el original o, si difiere, las diferencias quedan documentadas junto con ambas versiones para su análisis.
- **Alcance:** la prueba cubre la reproducibilidad de generador.py y la generación de resultados; no incluye reconstruir el grafo de conocimiento con build_graph.py, ya que este se entrega creado en entrega/grafos/.
- Ante cualquier error, se documenta la salida completa de la celda antes de intentar corregirla, para aislar qué parte de la entrega no resulta reproducible fuera del entorno original.

Para ejecutar el proyecto desde Kaggle, los principales comandos que se usan son los siguientes:

1. Verificar el contenido del Dataset

```
import os
```

```
print(os.listdir("/kaggle/input"))
```

Permite comprobar que el Dataset fue cargado correctamente en el Notebook.

2. Instalar las dependencias

```
!pip install -r /kaggle/input/datasets/julianriverag/codefest-ad-astra/entrega/requirements.txt
```

Instala las dependencias especificadas por el proyecto. En el proceso de configuración, las dependencias que faltaban (faiss-cpu y pysbd) quedaron disponibles en el entorno.

3. Cambiar al directorio del proyecto

```
import os
```

```
os.chdir("la dirección de donde está el proyecto")
```

```
print(os.getcwd())
```

Establece la carpeta del proyecto como directorio de trabajo para que las rutas relativas utilizadas por el programa funcionen correctamente.

4. Verificar el funcionamiento del script

```
!python generador.py --help
```

Comprueba que generador.py puede ejecutarse correctamente y muestra los argumentos disponibles.

5. Ejecutar el generador

```
!python generador.py \
```

```
--index base_vectorial/encoder_multilingual-e5-base/index.faiss \
```

```
--metadata base_vectorial/encoder_multilingual-e5-base/metadata.jsonl \
```

```
--consultas consultas.jsonl \
```

```
--out /kaggle/working/resultados_kaggle.jsonl
```

Ejecuta el pipeline de recuperación utilizando index.faiss (índice vectorial), metadata.jsonl (metadatos asociados al índice), consultas.jsonl (consultas de prueba), y resultados_kaggle.jsonl (archivo de resultados generado).